Data exploration with GeoPandas

CS424: Visualization & Visual Analytics

Fabio Miranda

https://fmiranda.me

Some slides based on Maryam Hosseini's urban informatics course

Interactive environments

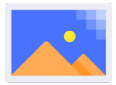
Data manipulation libraries

Low-level data manipulation libraries

Data formats

Tabular: Structured data in a **table format** (rows & columns).

Examples: CSV, Excel, relational databases.



Raster (image): Grid-based data where each cell (pixel) has a value.

Examples: Images (JPEG, PNG, TIFF), Satellite Data, Digital Elevation Models (DEMs).

Video: Temporal raster data: a sequence of raster images over time.
Examples: MP4, AVI, drone videos

Vector: Geospatial representations using **points, lines, polygons**.

Examples: Shapefiles (SHP), GeoJSON, OpenStreetMap data.

Data formats

Graph: Nodes and edges representing relationships/networks.

Examples: Social networks, transportation networks, citation networks.

3D Geometry – Explicit 3D shapes and surfaces used in modeling.

Examples: CAD models, 3D meshes (OBJ, STL), BIM models.

LiDAR Point Cloud – Sparse 3D spatial data collected via LiDAR sensors.

Examples: LAS, LAZ (georeferenced elevation/terrain scans).

Sound Data – Audio signals stored as waveforms or spectral data.

Examples: WAV, MP3, FLAC (raw or processed sound recordings).

Data sources

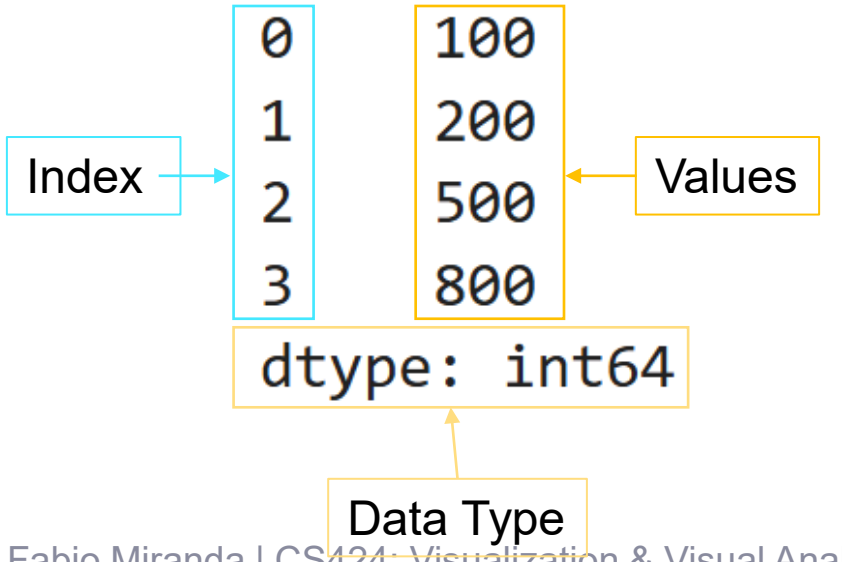
Category	Examples	Data Provided	Use Cases
Official Government (Federal, State, Local)	**Census Bureau, DOT**	Population, transportation, climate	Urban planning, policy analysis, infrastructure development
	EPA, NOAA	Pollution, environmental risks	Climate resilience, sustainability
Crowdsourced & Open	**OpenStreetMap (OSM)**	Roads, sidewalks, POIs	Accessibility mapping, navigation
	Project Sidewalk	Sidewalk accessibility	ADA compliance, walkability studies
Proprietary & Commercial	**Google Maps API, HERE**	Traffic, geolocation, POIs	Mobility analysis, retail planning
	Placer.ai, StreetLight Data	Human movement, GPS-based mobility	Transportation modeling, economic insights

Pandas

- [pandas] is derived from the term "panel data", an econometrics term for data sets that include observations over multiple time periods from multiple entities.
- Ranked among the most popular and widely used data wrangling tools
- Works with variety of formats
- Easy & quick visualization with integrated matplotlib
- Main data structures: **Series** and **DataFrame**

Series

```
my_series = pd.Series([100, 200, 500, 800])
my_series
```



DataFrame

A DataFrame is a multi-dimensional table made up of a collection of Series.

Series

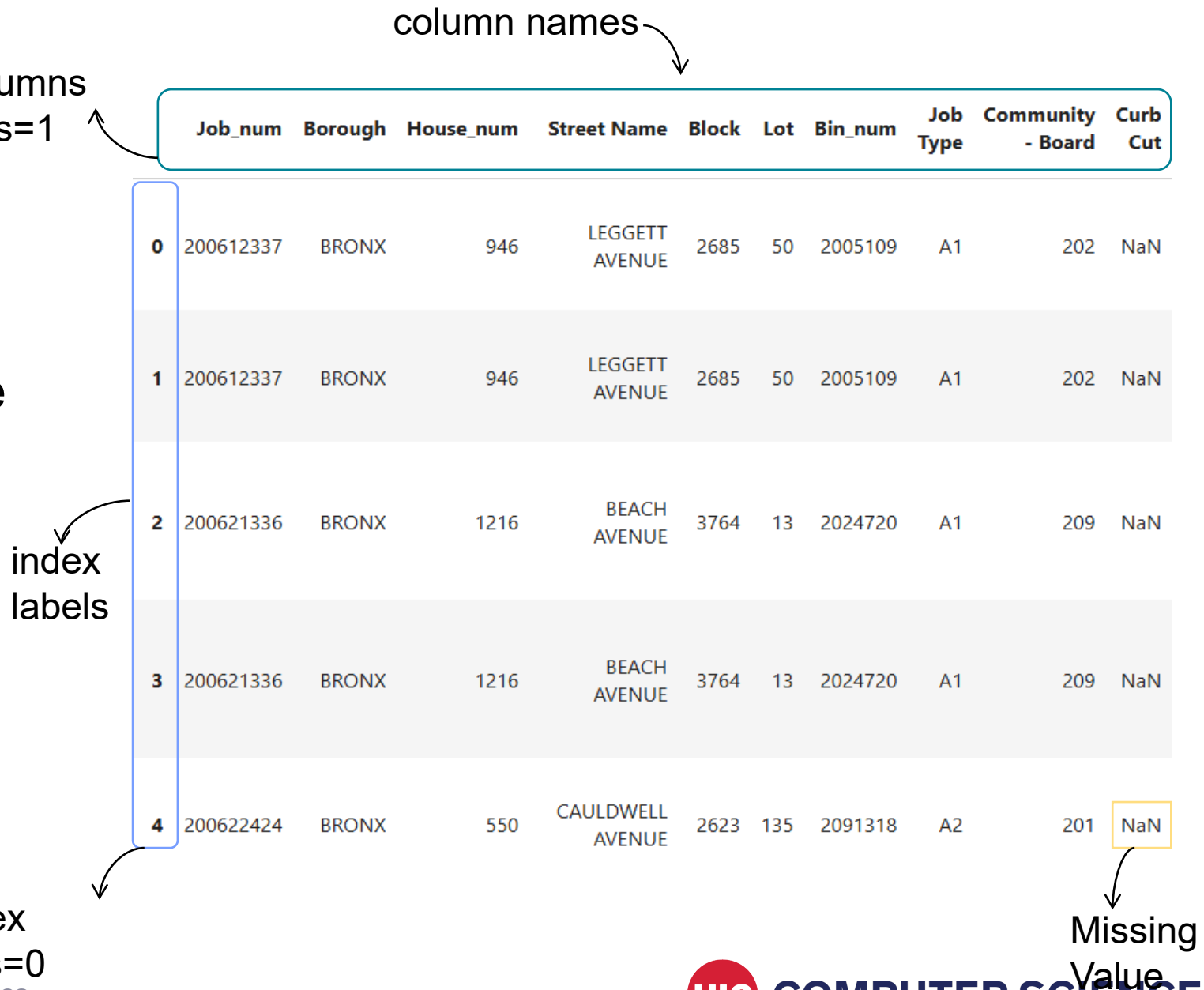
	Job_num	Borough	House_num	Street Name	Block	Lot	Bin_num	Job Type	Community - Board	Curb Cut
0	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
1	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
2	200621336	BRONX	1216	BEACH ENUE	3764	13	2024720	A1	209	NaN
3	200621336	BRONX	1216	EACH ENUE	3764	13	2024720	A1	209	NaN
19595	200621336	BRONX	1216	WELL ENUE	2623	135	2091318	A2	201	NaN

```
df['Block']

0         2685
1         2685
2         3764
3         3764
4         2623
         ...
19591      717
19592      717
19593     1096
19594      541
19595      541
Name: Block, Length: 19596, dtype: int64
```

DataFrame

A DataFrame is a multi-dimensional table made up of a collection of Series.



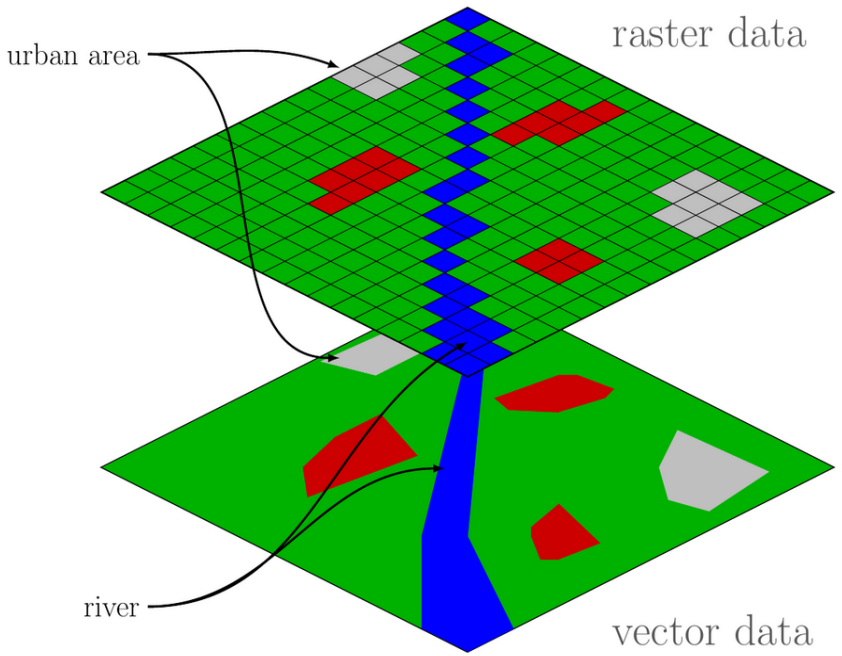
	Job_num	Borough	House_num	Street Name	Block	Lot	Bin_num	Job Type	Community - Board	Curb Cut
0	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
1	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
2	200621336	BRONX	1216	BEACH AVENUE	3764	13	2024720	A1	209	NaN
3	200621336	BRONX	1216	BEACH AVENUE	3764	13	2024720	A1	209	NaN
4	200622424	BRONX	550	CAULDWELL AVENUE	2623	135	2091318	A2	201	NaN

Geospatial data

- Data where the coordinate of the vertices point to the actual location on the surface of the earth
- **Projection**: the act of systematically transposing a 3D to a 2D object.
- Projection is a key concept of cartography, the art and science of making maps.

Raster data

- Rasters consist of a grid of cells (pixels) that represent spatial information.
- Their values and resolution determine the type of data captured:
  - **Single cell value:** Each pixel stores one value (e.g., elevation, temperature).
  - **Continuous data:** Represents smoothly varying phenomena (e.g., elevation models, climate surfaces).
  - **Categorical data:** Represents discrete classes (e.g., land cover, soil type).



Vector data

- Vectors consist of vertices (x, y values) that define spatial shapes.
- Their arrangement determines the type:
  - **Points:** Single x, y coordinate (e.g., POI sites)
  - **Lines:** Two or more connected vertices forming a continuous line (e.g., roads)
  - **Polygons:** Three or more vertices that are connected and closed (e.g., boundaries,)

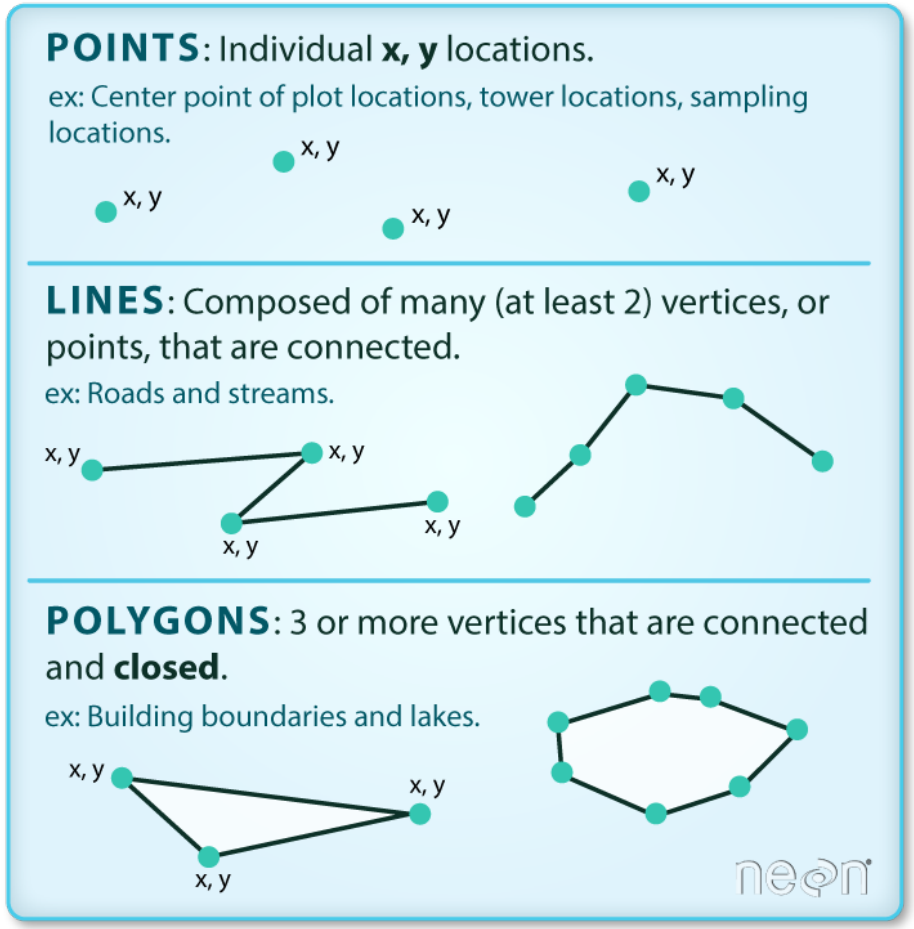


Image Source: National Ecological Observatory Network (NEON)

CRS: Coordinate Reference System

- Defines how coordinates map to locations on Earth.
- Essential for spatial analysis—datasets must share the same CRS to align correctly.
- Geographical Coordinate System
- Info about different projections: https://spatialreference.org/ref/epsg/

Two Main Types of CRS

Geographic CRS:

- Uses latitude and longitude (degrees).
- Defines global positions relative to the equator and prime meridian.
- Common example: **EPSG:4326 (WGS84)** (used in GPS).

Projected CRS:

- Converts Earth's curved surface into a **2D map** with X, Y coordinates.
- Uses **meters or feet** instead of degrees, improving spatial calculations.
- Introduces **distortions**, so different regions adopt their own standard projections.
- **Common example: EPSG:3857 (Web Mercator)** – widely used for web mapping but **distorts areas significantly near the poles**.

GeoPandas

- A Python package that handles geospatial data:

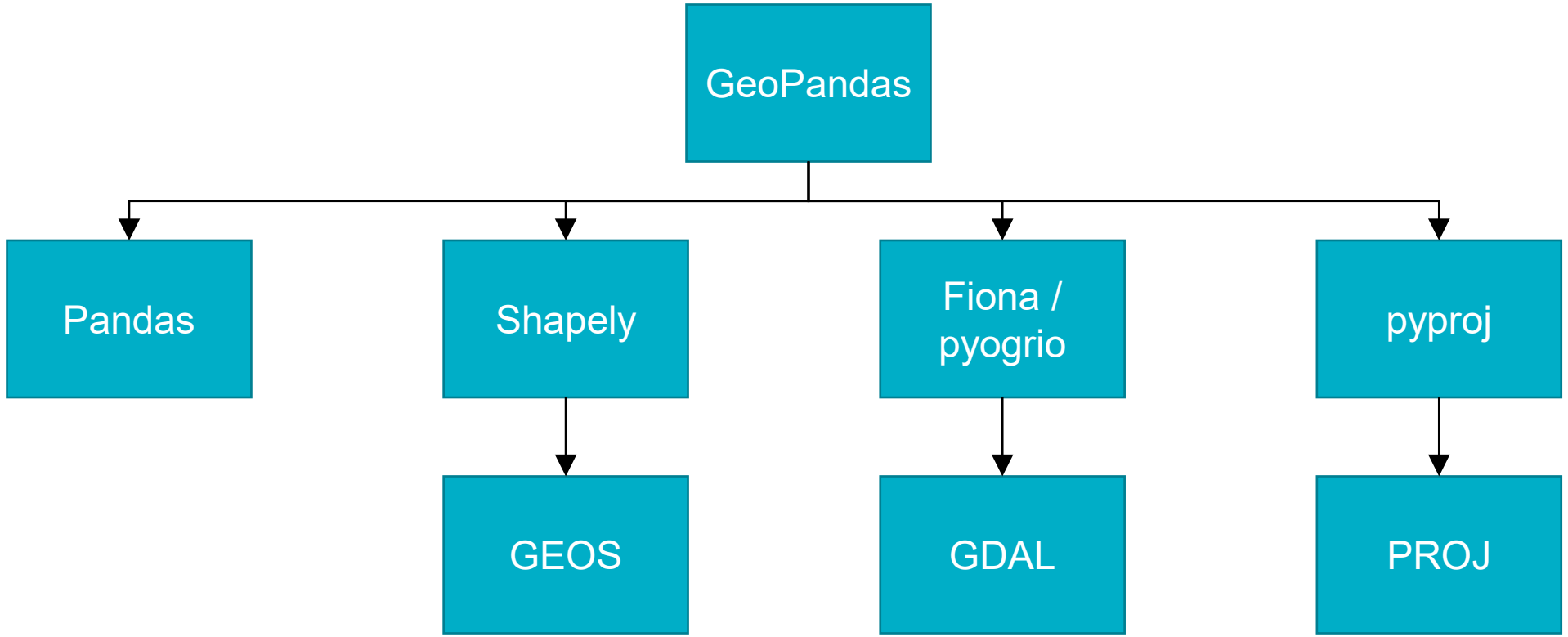
```
import geopandas as gpd
```

- GeoPandas, extends pandas by adding support for geospatial data.
- The core data structures in GeoPandas, like pandas, is GeoDataFrame that can store geometry columns and perform spatial operations.
- The geopandas.GeoSeries, a subclass of pandas.Series, handles the geometries.
- Combines the power of several libraries (Pandas, geos, shapely, gdal, pyproj, rtree, …)

GeoPandas

- Read and write several geo formats (Fiona, GDAL).
- Usual DataFrame manipulation.
- Element-wise spatial operations (intersection, union, difference, …)
- Re-project data.
- Visualize geometries.
- Advanced spatial operations: spatial joins and overlays.

GeoPandas


GeoPandas
Pandas
Shapely
Fiona / pyogrio
pyproj
GEOS
GDAL
PROJ

Shapely

- Python package for the manipulation of geometric objects.

```
1 from shapely.geometry import Point, LineString, Polygon
2
3 point = Point(1, 1)
4 line = LineString([(0, 0), (1, 2), (2, 2)])
5 poly = line.buffer(1)
```

```
1 line
```

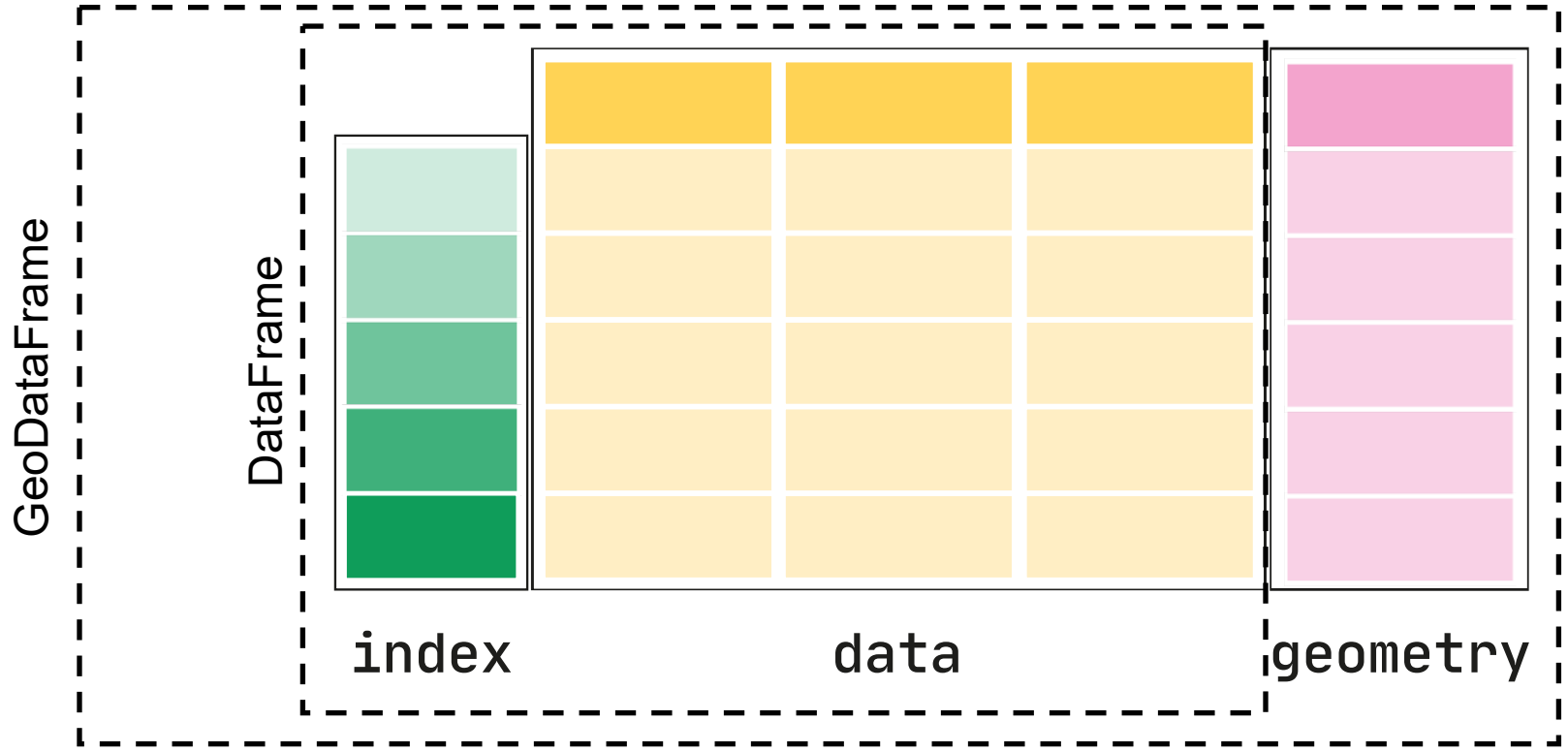
```
1 poly
```

```
1 poly.contains(point)
```

```
True
```

GeoPandas

- Core data structure in GeoPandas is the GeoDataFrame (subclass of Pandas' DataFrame).
  - It can store geometry columns and perform spatial operations.

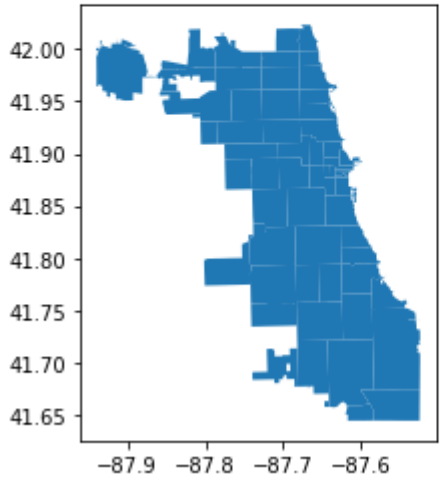


Reading and writing files

```
1 gdf = gpd.read_file('chicago.geojson')
```

```
1 gdf.plot()
```

```
<AxesSubplot:>
```




```
1 gdf
```

	objectid	shape_area	shape_len	zip	geometry
0	33	106052287.488	42720.0444058	60647	MULTIPOLYGON (((-87.67762 41.91776, -87.67761 ...
1	34	127476050.762	48103.7827213	60639	MULTIPOLYGON (((-87.72683 41.92265, -87.72693 ...
2	35	45069038.4783	27288.6096123	60707	MULTIPOLYGON (((-87.78500 41.90915, -87.78531 ...
3	36	70853834.3797	42527.9896789	60622	MULTIPOLYGON (((-87.66707 41.88885, -87.66707 ...
4	37	99039621.2518	47970.1401531	60651	MULTIPOLYGON (((-87.70656 41.89555, -87.70672 ...
...	...	...	...	...	...
56	57	155285532.005	53406.9156168	60623	MULTIPOLYGON (((-87.69479 41.83008, -87.69486 ...
57	58	211114779.439	58701.3253749	60629	MULTIPOLYGON (((-87.68306 41.75786, -87.68306 ...
58	59	211696050.967	58466.1602979	60620	MULTIPOLYGON (((-87.62373 41.72167, -87.62388 ...
59	60	125424284.172	52377.8545408	60637	MULTIPOLYGON (((-87.57691 41.79511, -87.57700 ...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-87.58592 41.75150, -87.58592 ...

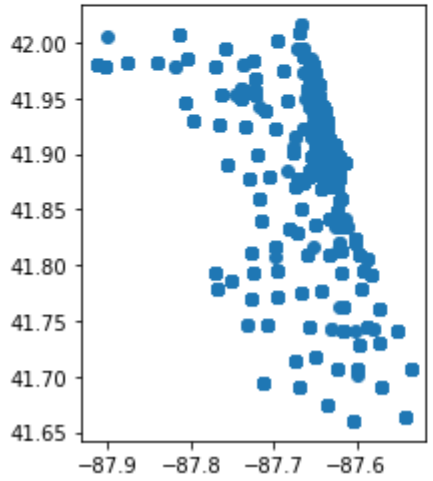
61 rows × 5 columns

Reading CSV file

```
1 df = pd.read_csv('data/Taxi_Trips.csv')
2 geometry = [Point(xy) for xy in zip(df['Pickup Centroid Longitude'], df['Pickup Centroid Latitude'])]
3 gdf = gpd.GeoDataFrame(df, geometry=geometry, crs=4326)
```

```
1 gdf.plot()
```

```
<AxesSubplot:>
```



Geometry from lon / lat (x / y)

Projections

```
1 gdf
```

	objectid	shape_area	shape_len	zip	geometry
0	33	106052287.488	42720.0444058	60647	MULTIPOLYGON (((-87.67762 41.91776, -87.67761 ...
1	34	127476050.762	48103.7827213	60639	MULTIPOLYGON (((-87.72683 41.92265, -87.72693 ...
2	35	45069038.4783	27288.6096123	60707	MULTIPOLYGON (((-87.78500 41.90915, -87.78531 ...
3	36	70853834.3797	42527.9896789	60622	MULTIPOLYGON (((-87.66707 41.88885, -87.66707 ...
4	37	99039621.2518	47970.1401531	60651	MULTIPOLYGON (((-87.70656 41.89555, -87.70672 ...
...	...	...	...	...	...
56	57	155285532.005	53406.9156168	60623	MULTIPOLYGON (((-87.69479 41.83008, -87.69486 ...
57	58	211114779.439	58701.3253749	60629	MULTIPOLYGON (((-87.68306 41.75786, -87.68306 ...
58	59	211696050.967	58466.1602979	60620	MULTIPOLYGON (((-87.62373 41.72167, -87.62388 ...
59	60	125424284.172	52377.8545408	60637	MULTIPOLYGON (((-87.57691 41.79511, -87.57700 ...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-87.58592 41.75150, -87.58592 ...

61 rows × 5 columns

Projecting to EPSG:3395

```
1 gdf = gdf.to_crs("EPSG:3395")
```

```
1 gdf
```

	objectid	shape_area	shape_len	zip	geometry
0	33	106052287.488	42720.0444058	60647	MULTIPOLYGON (((-9760228.181 5120114.708, -976...
1	34	127476050.762	48103.7827213	60639	MULTIPOLYGON (((-9765706.326 5120843.341, -976...
2	35	45069038.4783	27288.6096123	60707	MULTIPOLYGON (((-9772181.764 5118831.519, -977...
3	36	70853834.3797	42527.9896789	60622	MULTIPOLYGON (((-9759053.446 5115807.386, -975...
4	37	99039621.2518	47970.1401531	60651	MULTIPOLYGON (((-9763449.188 5116805.817, -976...
...	...	...	...	...	...
56	57	155285532.005	53406.9156168	60623	MULTIPOLYGON (((-9762139.685 5107055.000, -976...
57	58	211114779.439	58701.3253749	60629	MULTIPOLYGON (((-9760833.554 5096312.536, -976...
58	59	211696050.967	58466.1602979	60620	MULTIPOLYGON (((-9754228.915 5090933.835, -975...
59	60	125424284.172	52377.8545408	60637	MULTIPOLYGON (((-9749017.532 5101851.550, -974...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-9750019.820 5095367.709, -975...

61 rows × 5 columns

Projections

```
1 gdf
```

	objectid	shape_area	shape_len	zip	geometry
0	33	106			.67761 ...
1	34	127			.72693 ...
2	35	450			.78531 ...
3	36	708			.66707 ...
4	37	990			.70672 ...
...	...				...
56	57	155			.69486 ...
57	58	21			.68306 ...
58	59	211			.62388 ...
59	60	125424284.172	52377.8545408	60637	MULTIPOLYGON (((-87.57691 41.79511, -87.57700 ...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-87.58592 41.75150, -87.58592 ...

```
<Geographic 2D CRS: EPSG:4326>
Name: WGS 84
Axis Info [ellipsoidal]:
- Lat[north]: Geodetic latitude (degree)
- Lon[east]: Geodetic longitude (degree)
Area of Use:
- name: World.
- bounds: (-180.0, -90.0, 180.0, 90.0)
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

61 rows × 5 columns

Projecting to EPSG:3395

```
1 gdf = gdf.to_crs("EPSG:3395")
```

```
1 gdf
```

	objectid	shape				...metry
0	33	1060522				...976...
1	34	1274760				...976...
2	35	4506903				...977...
3	36	7085383				...975...
4	37	9903962				...976...
...	...					...
56	57	1552855				...976...
57	58	2111147				...976...
58	59	2116960				...975...
59	60	1254242				...974...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-9750019.820 5095367.709, -975...	

```
<Derived Projected CRS: EPSG:3395>
Name: WGS 84 / World Mercator
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: World between 80°S and 84°N.
- bounds: (-180.0, -80.0, 180.0, 84.0)
Coordinate Operation:
- name: World Mercator
- method: Mercator (variant A)
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

61 rows × 5 columns

Simple operations

- Accessing & plotting geometry area

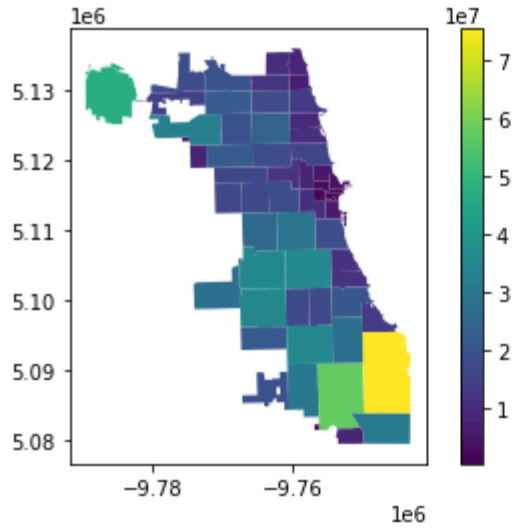
```
1 gdf.area
```

```
0     1.774279e+07
1     2.132685e+07
2     7.540024e+06
3     1.184732e+07
4     1.656003e+07
          ...
56    2.592093e+07
57    3.515998e+07
58    3.521726e+07
59    2.089192e+07
60    2.793029e+07
Length: 61, dtype: float64
```

```
1 gdf['area'] = gdf.area
```

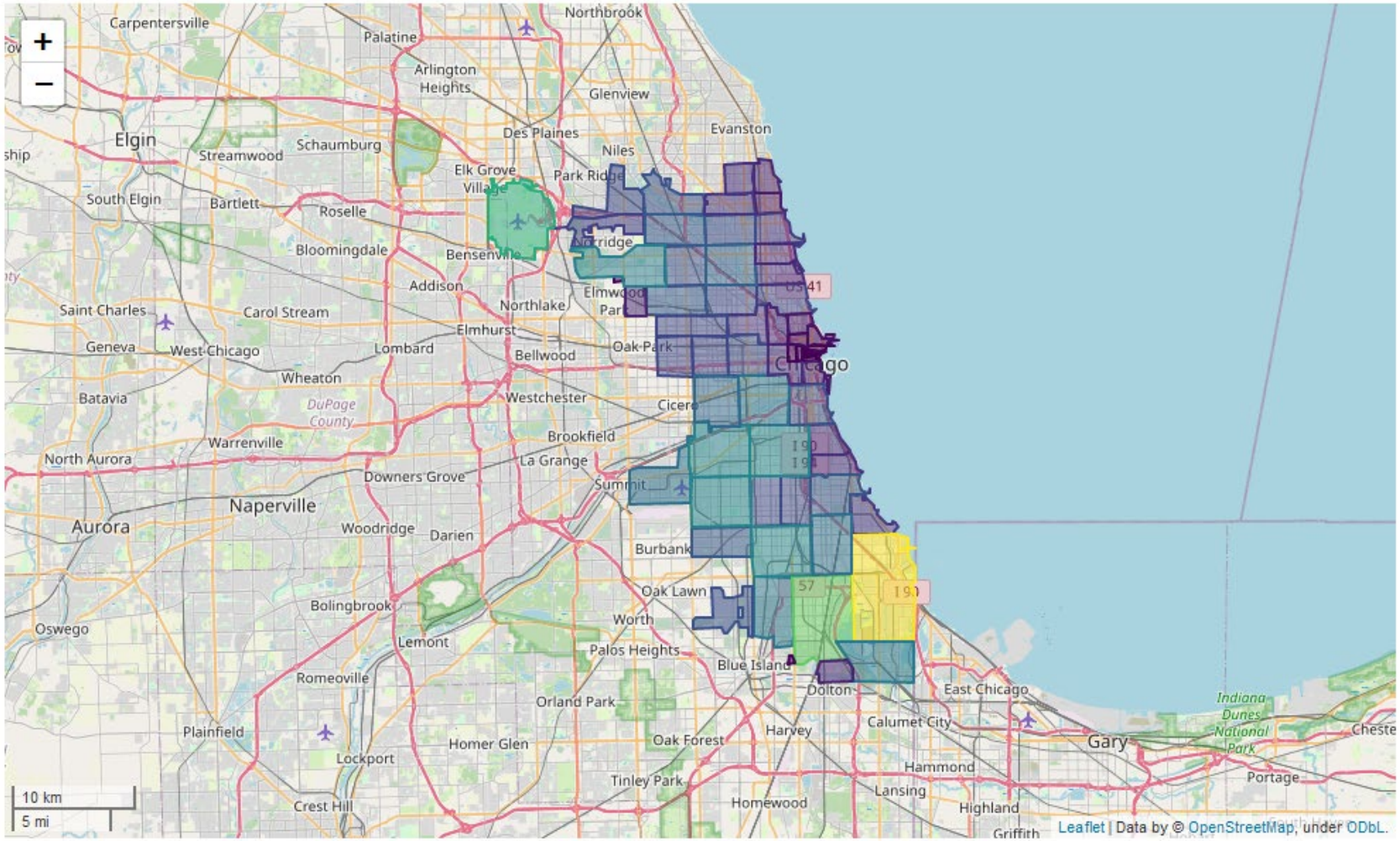
```
1 gdf.plot("area", legend=True)
```

```
<AxesSubplot:>
```



Simple operations

```
1 gdf.explore("area", legend=False)
```



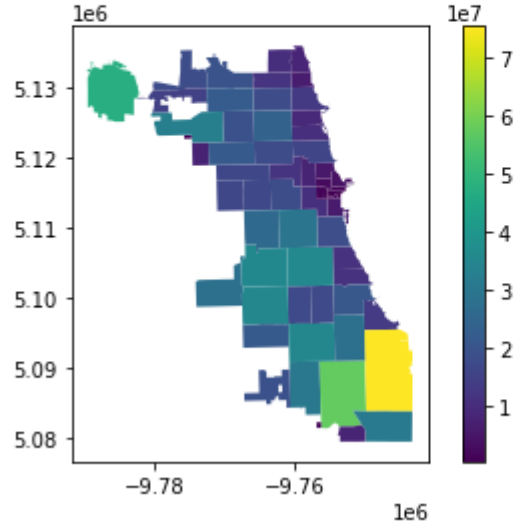
Simple operations

Changing geometry

```
1 gdf['area'] = gdf.area
```

```
1 gdf.plot("area", legend=True)
```

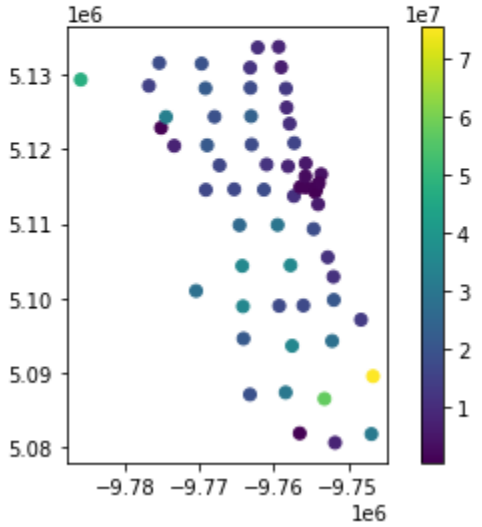
```
<AxesSubplot:>
```




```
1 gdf['centroid'] = gdf.centroid
```

```
1 gdf = gdf.set_geometry('centroid')
2 gdf.plot("area", legend=True)
```

```
<AxesSubplot:>
```



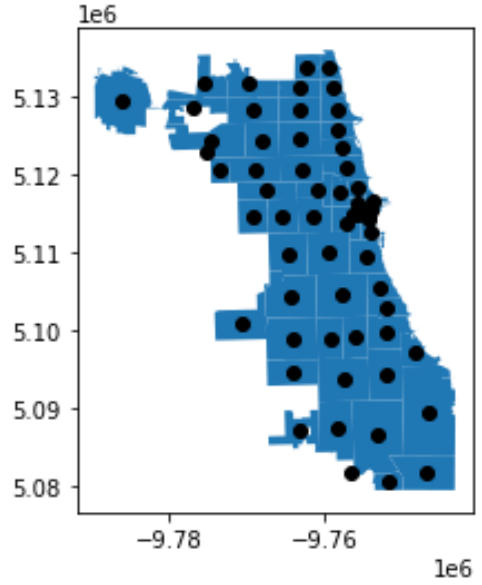
Simple operations

- Plotting centroids and zip areas:

```
1  gdf = gdf.set_geometry('geometry')
2  ax = gdf['geometry'].plot()
3  gdf['centroid'].plot(ax=ax, color="black")
```

Setting geometry back

```
<AxesSubplot:>
```



Geometry operations

```
1  gdf["convex_hull"] = gdf.convex_hull
```

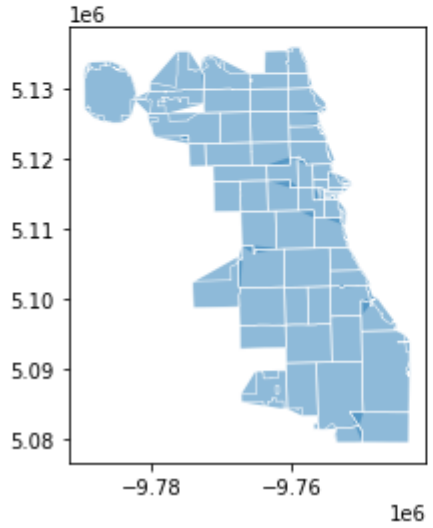
New column with convex hull

```
1  gdf['boundary'] = gdf.boundary
```

New column with boundary (i.e., outline)

```
1  ax = gdf["convex_hull"].plot(alpha=.5)
2  gdf['boundary'].plot(ax=ax, color="white", linewidth=.5)
```

```
<AxesSubplot:>
```



Geometry operations

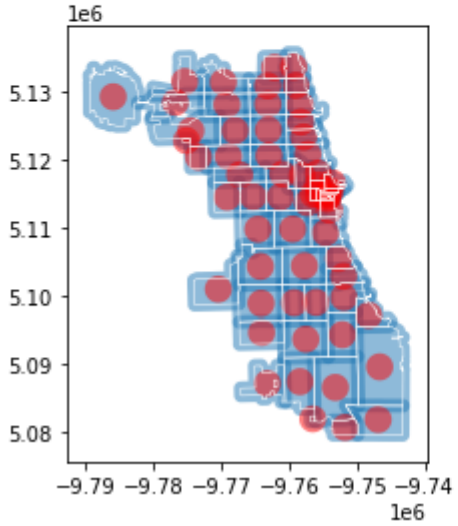
```
1 gdf["buffered"] = gdf.buffer(1000)
2 gdf["buffered_centroid"] = gdf["centroid"].buffer(2000)
```

Creating buffer around zip areas

Creating buffer around centroids

```
1 ax = gdf["buffered"].plot(alpha=.5)
2 gdf["buffered_centroid"].plot(ax=ax, color="red", alpha=.5)
3 gdf["boundary"].plot(ax=ax, color="white", linewidth=.5)
```

```
<AxesSubplot:>
```



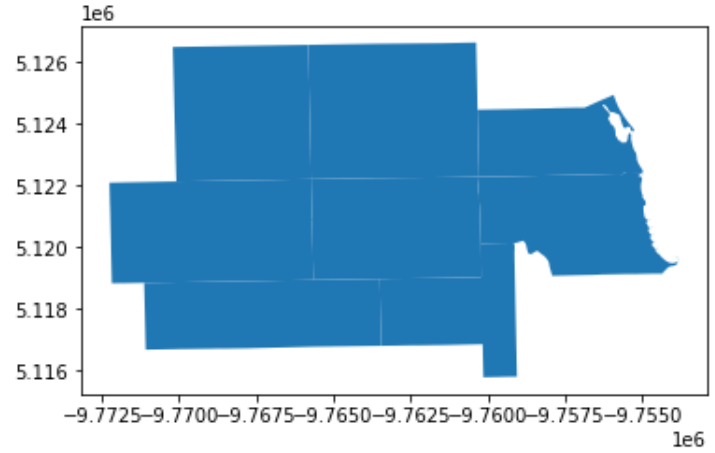
Geometry relations

- GeoPandas offers a series of operations for geometry relations:
  - Crosses, intersects, overlaps, covers, within, touches, …

```
1 selected = gdf.iloc[0]['geometry']
```

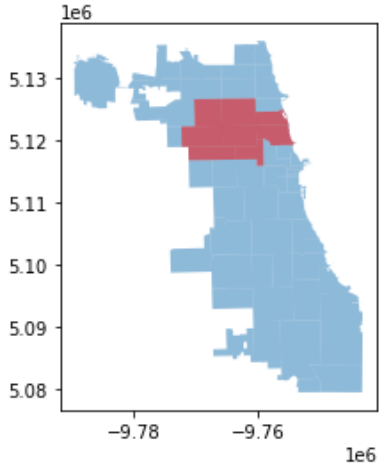
```
1 gdf[gdf['buffered'].intersects(selected)].plot()
```

```
<AxesSubplot:>
```




```
1 intersected = gdf[gdf['buffered'].intersects(selected)]
2 ax = gdf.plot(alpha=.5)
3 intersected.plot(ax=ax, color="red", alpha=.5)
```

```
<AxesSubplot:>
```



Geometry relations

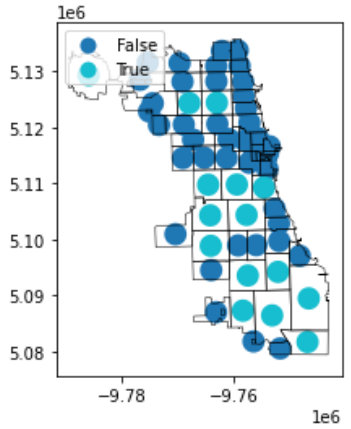
```
gdf["within"] = gdf["buffered_centroid"].within(gdf)
gdf["within"]
```

Whether buffered centroid is within zip area

```
0     False
1     False
2     False
3     False
4     False
      ...
56     True
57     True
58     True
59    False
60     True
Name: within, Length: 61, dtype: bool
```

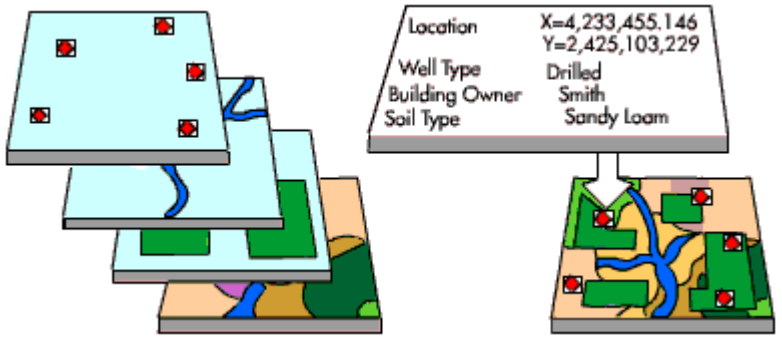
```
gdf = gdf.set_geometry("buffered_centroid")
ax = gdf.plot("within", legend=True, categorical=True, legend_kwds={'loc': "upper left"})
gdf["boundary"].plot(ax=ax, color="black", linewidth=.5)
```

```
<AxesSubplot:>
```



Spatial joins

- A spatial join combines two GeoDataFrames based on the spatial relationship between their geometries.
- Example: spatial join between point layer (e.g., taxi pickups) and a polygon layer (e.g., zip codes).



Spatial join

```
1  x_min, y_min, x_max, y_max = gdf.total_bounds
2
3  n = 1000
4
5  x = np.random.uniform(x_min, x_max, n)
6  y = np.random.uniform(y_min, y_max, n)
7
8  gdf_points = gpd.GeoDataFrame(geometry=gpd.points_from_xy(x, y), crs=gdf.crs)
9  gdf_points = gdf_points[gdf_points.within(gdf.unary_union)]
```

Bounds of GeoDataFrame

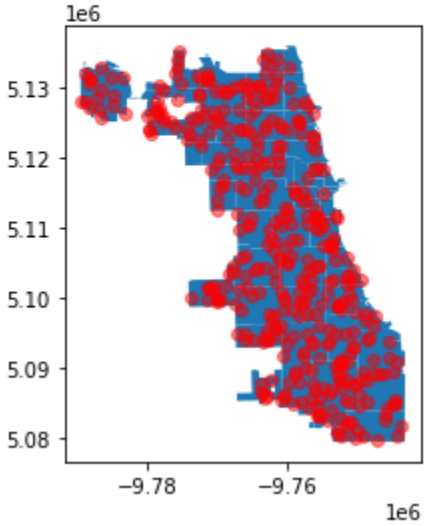
Sample size

Creating GeoDataFrame

Only keeping points within polygons

```
1  ax = gdf.plot()
2  gdf_points.plot(ax=ax, color="red", alpha=.5)
```

```
<AxesSubplot:>
```



Spatial join

- Spatial join: for each point, is it *within* what zip code?

```
1 gpd.sjoin(gdf_points, gdf, predicate='within')
```

	geometry	index_right	objectid	shape_area	shape_len	zip
1	POINT (-9774274.365 5122584.288)	51	52	194062612.162	77647.3180069	60634
133	POINT (-9774348.810 5124145.541)	51	52	194062612.162	77647.3180069	60634
164	POINT (-9776525.134 5125184.977)	51	52	194062612.162	77647.3180069	60634
207	POINT (-9770521.404 5126432.046)	51	52	194062612.162	77647.3180069	60634
253	POINT (-9777824.534 5126280.441)	51	52	194062612.162	77647.3180069	60634
...	...	...	...	...	...	...
578	POINT (-9761501.347 5118140.579)	3	36	70853834.3797	42527.9896789	60622
917	POINT (-9763366.621 5118966.647)	3	36	70853834.3797	42527.9896789	60622
714	POINT (-9755003.744 5114813.246)	40	26	4847124.8171	14448.1749926	60602
756	POINT (-9772253.752 5121497.783)	2	35	45069038.4783	27288.6096123	60707
894	POINT (-9759652.063 5132688.902)	8	1	49170578.9623	33983.9133065	60626

392 rows × 6 columns

Spatial join

- Grouping by zip code value to obtain number of points within that area.

```
1 result = gpd.sjoin(gdf_points, gdf, predicate='within').groupby('zip').count()
```

Grouping by zip code

```
1 result = result.filter(['geometry'])
2 result = result.rename(columns={'geometry': 'count'})
```

```
1 result
```

zip	count
60602	1
60605	1
60607	7
60608	12
60609	11
60610	3
60612	4
60613	4
60614	2
60615	8
60616	12
60617	20

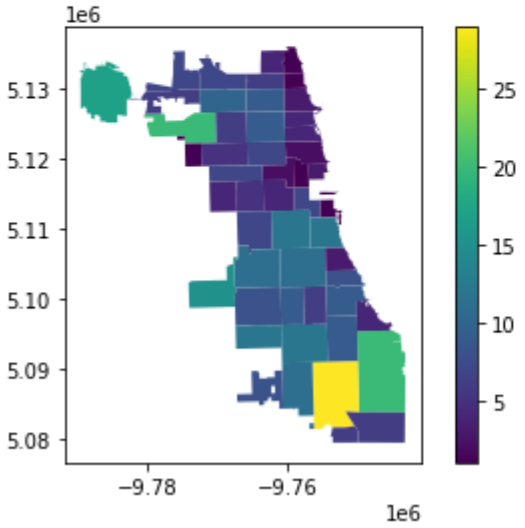
Spatial join

```
1  merged = pd.merge(result, gdf, right_on='zip', left_index=True)
```

Merging with previous zip code GeoDataFrame

```
1  merged = merged.set_geometry('geometry')
2  merged.plot('count', legend=True)
```

```
<AxesSubplot:>
```



Spatial join

Aggregating points over spatial regions:

1. Spatial join: map between point and polygon.
2. Group by: aggregate (sum, count, mean, …) by polygon.
3. Merge / join: map between aggregations and polygons.

Aggregation Bias

- Aggregation bias occurs when incorrect inferences are made from data that have been aggregated, or summarized.
- This includes making inferences about the characteristics of the parts of a whole based on the characteristics of the whole itself.

Assessing data quality

- Assessing data quality means looking at all the details provided about the data (including metadata, or “data about the data,” such as time and date of creation) and the context in which the data is presented.
- Good datasets will provide details about the dataset’s purpose, ownership, methods, scope, dates, and other notes.
- For online datasets, you can often find this information by navigating to the “About” or “More Information” web pages or by following a “Documentation” link.

CRAAP test: Assessing data quality

Currency	Is the information up-to-date? When was it collected / published / updated?
Relevancy	Is the information suitable for your intended use? Does it address your research question? Is there other (better) information?
Authority	Is the information creator reputable and has the necessary credentials? Can you trust the information?
Accuracy	Do you spot any errors? What is the source of the information? Can other data or research support this information?
Purpose	What was the intended purpose of the information collected? Are other potential uses identified?

Data checklist

Checklist	
Did you get all the necessary details about the data?	Don’t forget to obtain variable specifications, external data dictionaries, and referenced works.
Is the data part of a bigger dataset or body of research?	If yes, look for relevant specifications or notes from the bigger dataset.
Has the dataset been used before?	If it has and you’re using the data for an analysis, make sure your analysis is adding new insights to what you know has been done with the data previously.
How are you documenting your process and use of the data?	Make sure to keep records of licensing rights, communication with data owners, data storage and retention, if applicable.
Are you planning to share your results or findings in the future?	If yes, you’ll need to include your data dictionary and a list of your additional data sources.

Data checklist

Your answers to these questions can refine your analysis, prompt the search for additional data, or even inspire a new research question.

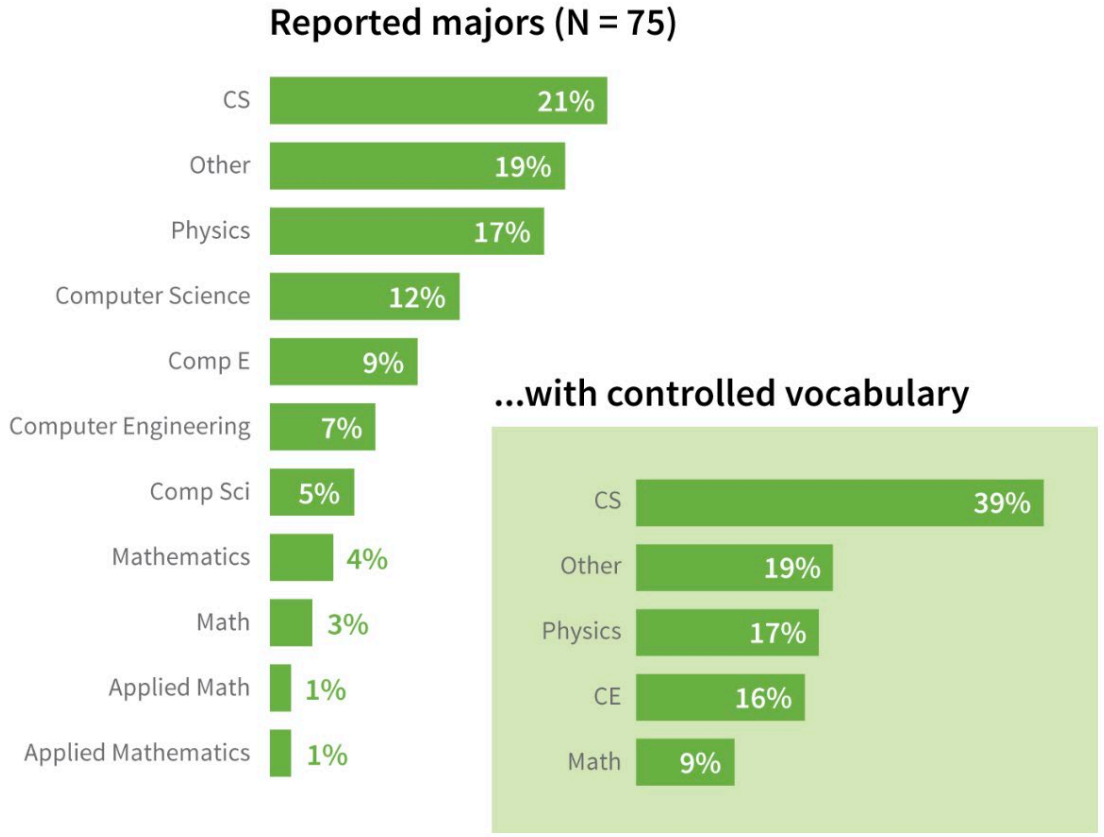
The checklist encourages you to document (a lot).

Careful documentation is essential for two key reasons:

1. It enables you to retrace your steps if you need to revisit or redo your analysis.
2. It serves as evidence for other researchers, ensuring transparency and allowing them to build on your findings.

Data cleaning

- Unit and unit conversions
- Type conversion
  - Inconsistent data types (like numbers stored as text) can lead to misinterpretations in visualizations.
  - Ensuring consistent data types prevents errors and maintains the integrity of your visual analysis
- Detecting anomalies
- Handing inconsistencies



Data cleaning

- **Range Checks**
- **Spell Check**
- Validity checks
- If the number of incorrect or missing values in a row /column is greater than the number of correct values, it is recommended to exclude that row/column.